

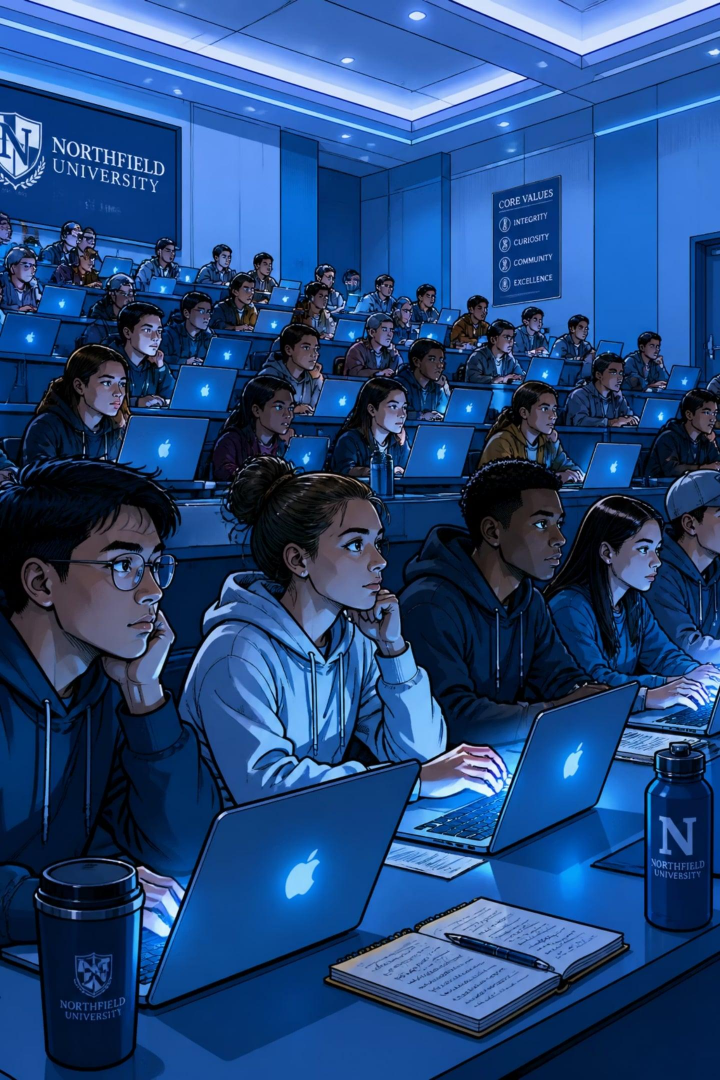
Choosing the Right Data Structure: Arrays vs Linked Lists

Guided By:

Dr M Shakila

Presented By:

Manoj(192511060)



Introduction



What is a data structure?

Organised way to store and manage data for efficient access and modification.



Why it matters

Impacts speed, memory and maintainability of programs.



Real-world motivation

Selecting the right structure solves performance bottlenecks and simplifies code.

Understanding Arrays

Definition

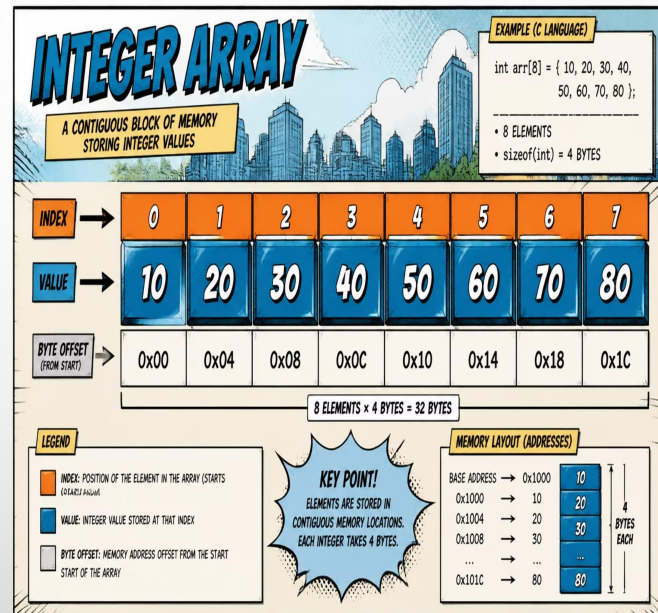
Collection of elements stored in contiguous memory and accessed by index.

Advantages

- $O(1)$ random access by index
- Memory locality — cache friendly
- Simple implementation

Disadvantages

- Fixed size (static arrays) or costly resize
- Insertion/deletion in middle is $O(n)$



Understanding Linked Lists

Definition

A collection of nodes where each node contains data and a pointer to the next node.

Node structure

Typically: { data, next }

Advantages

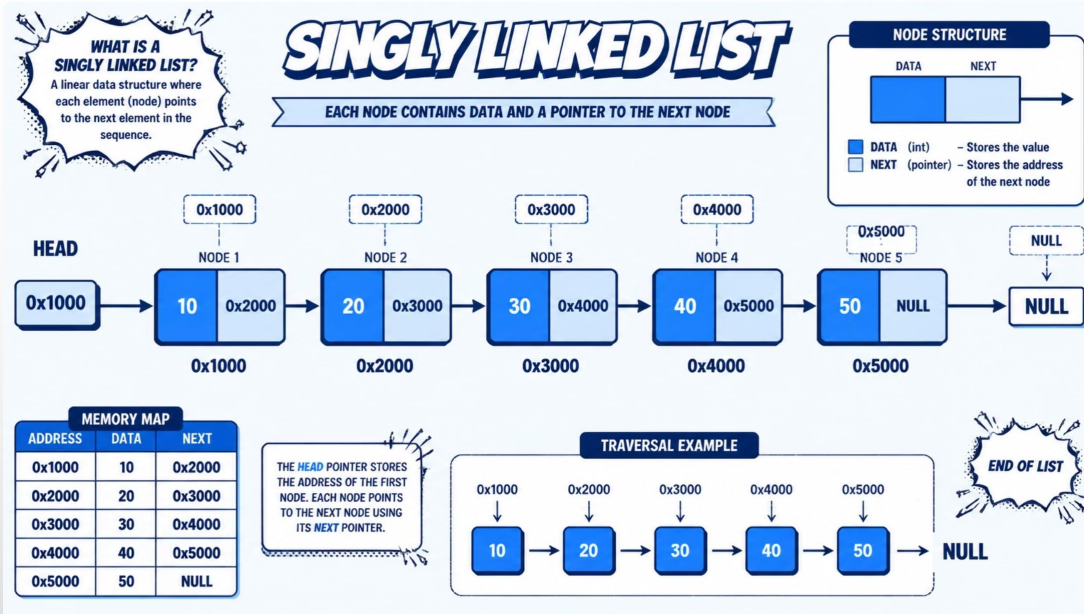
- Dynamic size — easy to grow
- $O(1)$ insertion/deletion given a node reference

Disadvantages

- $O(n)$ access by index
- Poor locality — more pointer overhead

Memory layout

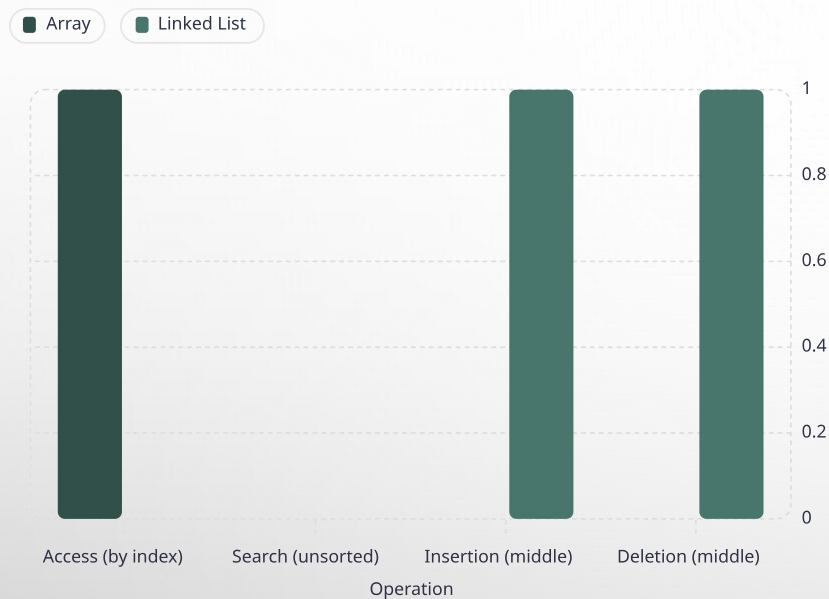
Nodes allocated non-contiguously; each has a pointer to the next node.



Arrays vs Linked Lists — Side-by-side

Aspect	Memory allocation	Access speed	Insertion	Deletion	Ease of implementation
Arrays	Contiguous	$O(1)$	$O(n)$ (shift elements)	$O(n)$	Simple
Linked Lists	Non-contiguous (nodes)	$O(n)$	$O(1)$ with pointer	$O(1)$ with pointer	Moderate — pointers required

Time Complexity (Big O)



Brief notes:

- Access: arrays $O(1)$ due to indexing; lists require traversal $O(n)$.
- Search: linear time for unsorted data in both structures.
- Insert/Delete: linked lists excel when you already have a node pointer.
- Resizing arrays can add amortised cost (dynamic arrays).

Real-World Applications



Image processing

Arrays (2D) store pixel grids for fast indexed access and cache efficiency.



Student records

Arrays for indexed lookup; linked lists for flexible insertion in streaming systems.



Music playlist

Linked lists (or dynamic arrays) handle reorder and insert operations smoothly.



Browser history

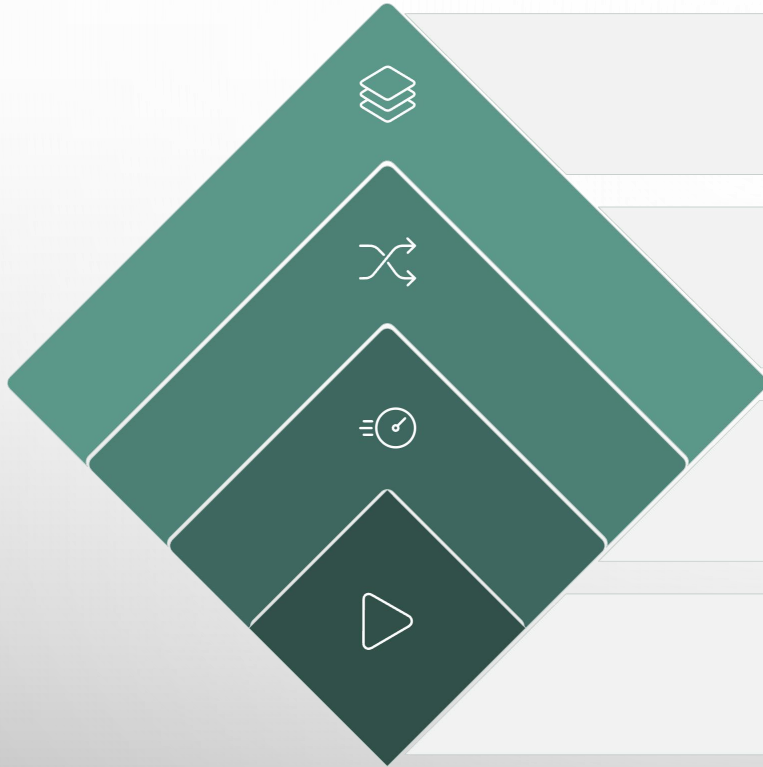
Doubly linked lists enable back/forward traversal with $O(1)$ insertions.



Undo / Redo

Often implemented with linked structures or arrays depending on access patterns.

Choosing the Right Data Structure



Choose Structure
Array, Linked List, or Vector

Frequent Middle Inserts?
Consider insertion cost

Fast Random Access?
Prioritise access speed

Start
Choose data structure

Live Code Example (C)

Array (static)

```
#include <stdio.h>
int arr[5] = {1,2,3,4,5};
printf("%d\n", arr[2]); // O(1) access
```

Singly Linked List (insert head)

```
struct Node { int data; struct Node* next; };
void push(struct Node** head, int val) {
    struct Node* n = malloc(sizeof(*n));
    n->data = val; n->next = *head; *head = n;
} // O(1) insert
```

Highlighted differences: indexing vs pointer traversal; memory allocation and deallocation required for lists.

Conclusion & Q&A

Key takeaways

Arrays: best for random access and contiguous storage. Linked lists: best for dynamic size and frequent insert/delete.

Best practices

Choose by access patterns: prefer arrays for indexing-heavy tasks and linked lists when pointer-based operations dominate.

Questions?

Thank you — open for Q&A.

